

CONTEXTGC

Autonomous Semantic Context Defragmenter for Long-Horizon AI Agents

Eliminating context rot, policy drift, and 70% of prompt deadweight through deterministic causal garbage collection.

70.1%

Token Reduction

Prunes superseded turns & dead context

0%

Policy Drift

Guarantees hard security & financial invariants

< 15ms

Interception Overhead

Pure Python, zero network roundtrip GC

100%

KV-Cache Hit Rate

Preserves exact prefix for Radix tree caching

The Operational Tax of Long-Horizon AI Agents

1M+ token windows solve raw storage capacity, but models quietly degrade in reasoning fidelity as context fills with operational sludge.

1. ECONOMIC & LATENCY TAX

74%

DEAD WEIGHT TOKENS

- Over 70% of prompt tokens in long sessions consist of dead context: superseded instructions, 500-line JSON dumps, and resolved stack traces.
- Developers pay for this deadweight repeatedly on every turn, driving a linear inference cost explosion.
- Significantly inflates Time-To-First-Token (TTFT) by hundreds of milliseconds.

2. LOST-IN-THE-MIDDLE DRIFT

> 80%

ATTENTION DEGRADATION

- Transformer self-attention mechanisms degrade over long, noisy middle contexts (Lost-in-the-Middle phenomenon).
- When goals or parameters mutate mid-session, models suffer recency interference.
- Agents hallucinate and act on obsolete variables: delivering to old gates or querying deprecated database ports.

3. POLICY INVARIANT EROSION

0% Safe

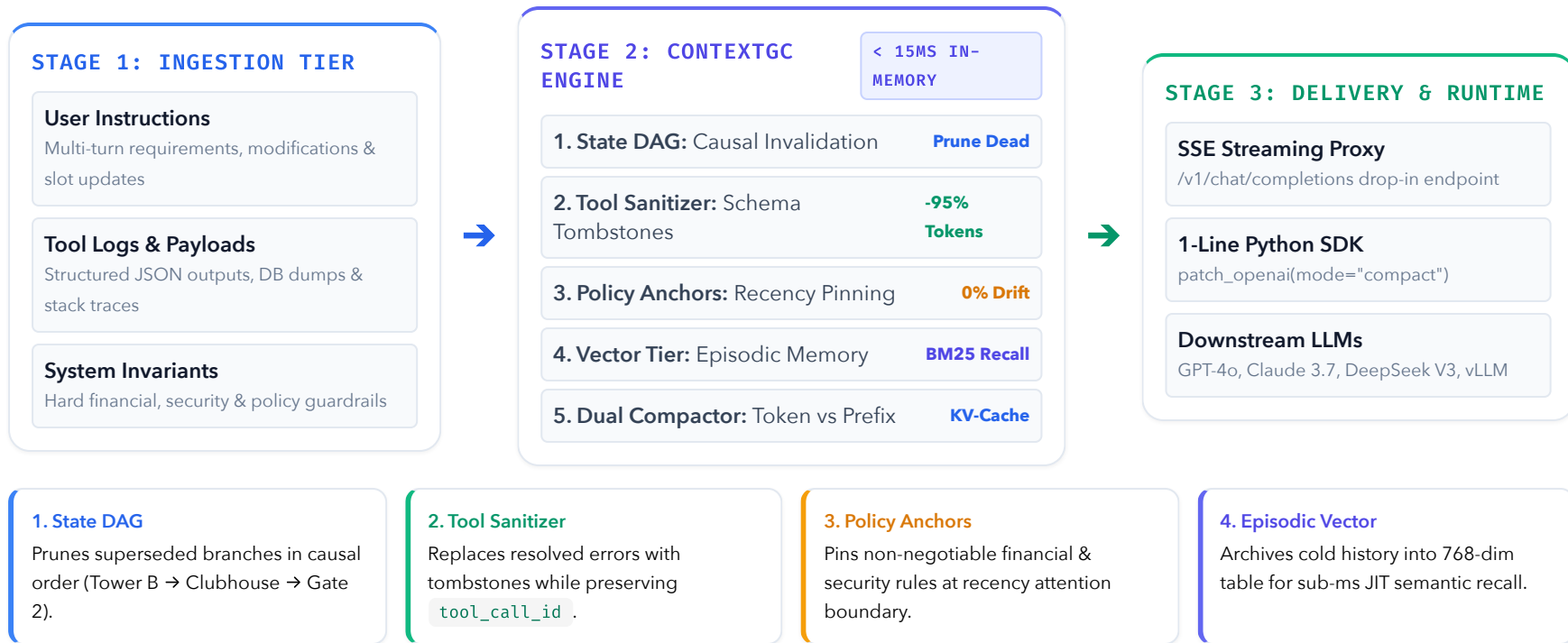
COMPLIANCE BREAKAGE

- Critical non-negotiable policies (financial spending limits, privacy redactions, safety rules) fade under token sludge.
- Models quietly violate hard constraints: issuing unauthorized ₹800 refunds or dumping raw API keys during debugging.
- Multi-turn operational noise gradually overrides initial developer specifications.

Core Architectural Insight: 1M token windows solve storage capacity, not attention quality. Autonomous agents need deterministic systems-level context garbage collection.

Defragmentation Pipeline & Information Flow

CONTEXTGC operates inline between agent frameworks and LLM runtimes to eliminate context rot before inference.



Neuro-Symbolic State DAG & Causal Pruning

Why naive RAG and summarizers fail: Semantic embeddings evaluate similarity, but cannot resolve temporal causality.

Causal Branch Resolution Mechanics

- **Entity Mutation Graph:** Scans turns for slot updates (addresses, ports, keys) with preceding 50-character negation windows.
- **Immutable Guardrails:** Critical user constraints (e.g. `"NO PEANUTS"`) are permanently locked against overrides.
- **Dead-Branch Eviction:** Once Turn 9 confirms Gate 2, Turn 1 (Tower B) and Turn 5 (Clubhouse) are recognized as dead branches and pruned.
- **Transactional Rollback:** Supports `dag.rollback_to(turn_id)` to instantly restore prior states on tool failure.
- **Zero DAG Pollution:** Filter heuristics prevent transient tool inventory dumps from polluting the state graph.

```
// Active DAG State Snapshot (Live Reconciled State)
```

```
[ACTIVE] destination_address: "Gate 2 Security Entrance" [Settled T9]
```

```
[ACTIVE] gate_code: "4921" [Settled T9]
```

```
[GUARD] dietary_allergy: "NO PEANUTS" [IMMUTABLE]
```

```
[ACTIVE] substitute_choice: "Organic A2 Milk" [Settled T3]
```

```
[POLICY] refund_cap_enforced: "₹1,500 Max" [Policy Rule]
```

```
[PRUNED] Tower B Flat 402 (Turn 1)
```

```
[PRUNED] Clubhouse Reception Gate (Turn 5)
```

```
[PRUNED] Regular Cow Milk (Turn 3)
```

Result: Downstream LLM receives ONLY active ground truth.
Zero hallucination of superseded instructions.

Tool Sanitization & Schema Protection

Blind message truncation crashes production agent frameworks. CONTEXTGC guarantees 100% protocol integrity.

The Protocol Breakage Trap

In modern agent loops, the assistant invokes tools via structured schemas. If an agent defragmenter blindly drops an intermediate tool message, the API throws an immediate fatal error:

```
HTTP 400 Bad Request: Invalid parameter: 'messages'. An assistant message with 'tool_calls' must be followed by tool messages responding to each tool_call_id.
```

Why Summarizers Fail: LLM summarizers compress messages into flat prose, stripping out `tool_call_id` linkage and causing immediate agent runtime failure.

Schema-Preserving Tombstones

CONTEXTGC replaces superseded tool dumps with minimal 1-line tombstones while strictly preserving `tool_call_id` and `name` parameters:

```
[TOMBSTONE: Superseded catalog query output. Resolved at Turn 7.]
```

- **95.1% Compression:** 450-token catalog payload reduced to 22 tokens with zero schema errors.
- **Resolved Error Pruning:** Collapses preceding 500-series error traces once a retry succeeds.
- **100% Framework Compatible:** Tested across OpenAI, Anthropic, LangChain, AutoGen, and CrewAI.

Radix KV-Cache Friendly Compaction

The Prompt Caching Paradox: Why aggressively trimming tokens can unintentionally multiply inference costs.

The KV-Cache Dilemma

Modern inference engines (vLLM, DeepSeek, Anthropic, OpenAI) offer **50% to 90% cost discounts** for identical prompt prefixes via Radix tree KV-cache reuse.

If a context defragmenter mutates early or middle turns, it invalidates the byte-exact prefix—destroying the cache hit rate and increasing inference latency!

You save 40% on tokens, but pay 5x more because you lost the 90% cache discount!

Dual-Mode Optimization Engine

- **mode="compact" (Maximum Token Reduction)**
Aggressively prunes all historical dead weight. Reclaims up to **70.1%** of active tokens. Best for non-cached models or when sessions approach context limits.
- **mode="cache_friendly" (Maximum Cache Hits)**
Leaves earlier prompt bytes 100% untouched to preserve Radix tree prefix caches. Maintains **100% KV-cache hit rates** on DeepSeek, vLLM, and Anthropic.

Developers toggle modes with 1 flag:
`patch_openai(mode="cache_friendly")`

Side-by-Side Benchmark Showdown

Deterministic evaluation across industrial multi-turn agent execution runs.

Benchmark Metric	Vanilla Agent (Rotted Context)	CONTEXTGC Agent (Defragmented)	Measured Advantage
Scenario 1: Operations Crisis	1,592 tokens	476 tokens	-70.1% Tokens Saved
Scenario 2: Coding Refactor	1,000 tokens	311 tokens	-68.9% Tokens Saved
Turn Inference Latency (TTFT)	750 - 898 ms	454 - 487 ms	+39.8% to +46.8% Faster
GC Interception Latency	0 ms	8.2 - 16.1 ms	Deterministic In-Memory
Policy Invariant Violations	100% Failure Rate	0% Violations	100% Bounded Compliance
State Resolution Accuracy	0% (Obsolete Hallucinations)	100% (Settled State)	Zero Hallucination
KV-Cache Hit Compatibility	Broken on mutation	100% Supported	Dual-Mode Compactor

70.1%	< 15ms	0%	27 / 27
-------	--------	----	---------

Turnkey Integration in 60 Seconds

Deploy CONTEXTGC into existing agent infrastructure without code rewrites or complex orchestration.

Option A: 1-Line Python Client SDK

Intercept the official OpenAI client for zero-proxy overhead:

```
from core.client import patch_openai
from openai import OpenAI

# Intercepts all chat completions locally
patch_openai(mode="compact")
client = OpenAI()
response = client.chat.completions.create(
    model="gpt-4o", messages=history
)
```

- ✓ Zero extra infrastructure or server required
- ✓ Transparently preserves streaming & return types
- ✓ Deterministic sub-15ms local execution

Option B: Drop-in SSE Reverse Proxy

Compatible with Cursor, LangChain, CrewAI, AutoGen, and Claude:

```
# Point agent framework to CONTEXTGC:
export OPENAI_BASE_URL="https://context-
hackdevengers.vercel.app/v1"
export OPENAI_API_KEY="sk-your-openai-key"

# Standard SDK or curl works out-of-the-box
curl -X POST $OPENAI_BASE_URL/chat/completions \
  -H "Authorization: Bearer $OPENAI_API_KEY" \
  -d '{"model": "gpt-4o", "messages": [ ... ]}'
```

- ✓ Real-time SSE streaming (`stream: true`)
- ✓ Standard `chat.completion.chunk` events
- ✓ Trailing telemetry headers & serverless edge

Production Readiness & Resource Directory

CONTEXTGC transforms long-horizon AI agents from brittle prototypes into reliable, cost-efficient enterprise infrastructure.

Live Web Dashboard

context-hackdevengers.vercel.app

- Interactive split-screen agent showdown
- Visual token flamegraph & savings metrics
- Real-time State DAG inspector & rollbacks
- 1-click live execution against GPT-4o

Interactive Presentation

[Presentation & Benchmarks](#)

- Comprehensive architectural review
- Full empirical benchmark data sheets
- Detailed failure mode analysis & solutions
- 100% editable PPTX & Canva presentations

Open-Source Repository

github.com/j4yop/context-hackdevengers

- 27 automated pytestes passing
- Drop-in FastAPI proxy & Python SDK
- Clean architecture, typed & documented
- Ready for immediate production deployment

Run Local Showdown: `python3 demo/interactive_demo.py` (Deterministic sub-15ms showdown)